

C++ ile Nesne Yönelimli Programlama Tabanlı İşlem Bellek İzleme ve Güvenlik Tespit Sistemi

Dita Salsabila Lubis

Öğrenci

Bilgisayar Mühendisliği Bölümü

Kastamonu university, Kastamonu, Türkiye

Email: ditasalsabilalubis@gmail.com

Özet—Modern bilişim ortamlarında sistem güvenliği ve kaynak izleme, sistem kararlılığını sağlamak ve kötü amaçlı veya anormal davranışları tespit etmek açısından kritik öneme sahiptir. Saldırı tespit sistemleri, bir sistem içerisindeki şüpheli aktiviteleri belirleyerek siber güvenlik çerçevelerinde önemli bir rol oynamaktadır [2]. Bu çalışma, C++ programlama dili kullanılarak Nesne Yönelimli Programlama (OOP) prensipleri ile geliştirilmiş basitleştirilmiş bir İşlem Bellek İzleme ve Güvenlik Tespit Sistemi sunmaktadır. Sistem; işlem takibi, bellek kullanımı analizi ve önceden tanımlanmış eşik değerlerine dayalı anomali tespiti gerçekleştirmektedir. Önerilen mimari; kapsülleme, sınıflar arası etkileşim ve görev ayrımı gibi modüler tasarım ilkelerini göstermektedir. Sistem basitleştirilmiş olmasına rağmen, işletim sistemi araçları ve güvenlik yazılımlarında kullanılan temel kavramları yansıtmaktadır. C++ dili, performansı ve düşük seviyeli bellek kontrolü sağlaması nedeniyle sistem seviyesinde yaygın olarak kullanılmaktadır [3].

Anahtar Kelimeler— İşlem İzleme, Bellek Kullanımı, Güvenlik Tespiti, C++, Nesne Yönelimli Programlama, Saldırı Tespit Sistemi

I. GİRİŞ

Modern işletim sistemleri, süreçlerin etkin bir şekilde izlenmesi ve sistem kaynaklarının yönetilmesi için gelişmiş mekanizmalara ihtiyaç duymaktadır. Yetersiz izleme mekanizmaları bellek sızıntılarına, performans düşüşüne ve kötü amaçlı yazılım faaliyetlerine yol açabilir.

Saldırı tespit sistemleri (Intrusion Detection Systems - IDS), ilk olarak Denning tarafından bilgisayar sistemlerinde yetkisiz erişim ve anormal davranışların tespiti için bir model olarak tanımlanmıştır [2]. Bu sistemler günümüzde antivirüs yazılımları ve uç nokta koruma sistemlerinin temelini oluşturmaktadır.

İşlem izleme, CPU ve bellek gibi kaynakların işletim sistemi çekirdeği tarafından yönetildiği temel bir işletim sistemi kavramıdır [4]. Windows Görev Yöneticisi veya Linux “top” aracı gibi sistemler, gerçek zamanlı performans analizi sağlar.

Bu çalışma, C++ ve Nesne Yönelimli Programlama kullanarak aşağıdaki hedeflere odaklanan bir simülasyon sunmaktadır:

- İşlem takibi
- Bellek kullanımı izleme

- Kural tabanlı anomali tespiti

- Uyarı üretme

C++ dili, sistem seviyesinde verimliliği nedeniyle tercih edilmiştir [3].

II. İLGİLİ ÇALIŞMALAR

İşlem izleme ve saldırı tespit sistemleri siber güvenlik literatüründe geniş bir araştırma alanına sahiptir.

Axelsson, saldırı tespit sistemlerini anomali tabanlı ve imza tabanlı yöntemler olarak sınıflandıran bir taksonomi önermiştir [1]. Anomali tabanlı sistemler normal davranıştan sapmaları tespit ederken, imza tabanlı sistemler bilinen saldırı desenlerine dayanır.

Denning’in modeli, bilgisayar sistemlerinde istatistiksel anomali tespiti kavramını ortaya koymuştur [2]. Bu çalışmalar günümüzde kullanılan güvenlik sistemlerinin temelini oluşturmaktadır.

C++, sistem kaynaklarını doğrudan yönetebilme yeteneği sayesinde performans kritik uygulamalarda yaygın olarak kullanılmaktadır [3].

III. SİSTEM TASARIMI

A. Tasarım Genel Bakış

ÖSistem tasarımı, işletim sistemlerinin temel prensiplerinden olan süreç yönetimi ve kaynak tahsisi kavramlarından esinlenmiştir [4].

Sistem dört ana bileşenden oluşmaktadır:

- Process (İşlem)
- MemoryMonitor (Bellek İzleyici)
- SecurityDetector (Güvenlik Algılayıcı)
- SystemManager (Sistem Yöneticisi)

Her bileşen ayrı bir sınıf olarak Nesne Yönelimli Programlama prensiplerine göre tasarlanmıştır.

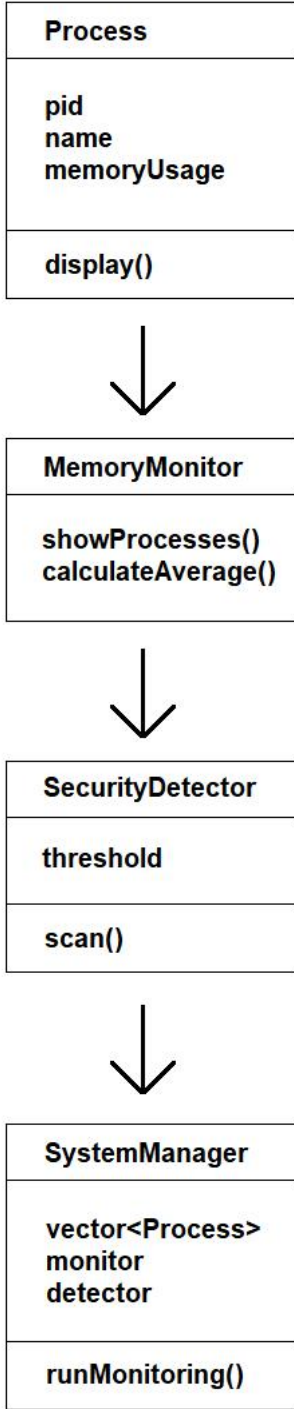


Fig. 1. Önerilen işlem izleme ve güvenlik tespit sisteminin mimarisi.

Fig. 1, önerilen izleme aracının genel sistem mimarisini göstermektedir. Sistem; Process, MemoryMonitor, SecurityDetector ve SystemManager olmak üzere dört ana bileşenden oluşmaktadır. Her bileşen Nesne Yönelimli Programlama prensiplerine uygun şekilde ayrı sınıflar olarak tasarlanmıştır.

C. Sınıf Açıklamaları

1) Process Sınıfı

PID, isim ve bellek kullanımı gibi işlem bilgilerini kapsüller. Veri gizleme ve modülerliği sağlar.

2) MemoryMonitor Sınıfı

Aşağıdaki işlemlerden sorumludur:

- İşlem verilerini görüntüleme
- Ortalama bellek kullanımını hesaplama

3) SecurityDetector Sınıfı

Kural tabanlı bir tespit mekanizması uygular. Eşik değeri aşan işlemler şüpheli olarak işaretlenir.

4) SystemManager Sınıfı

Sistemin kontrol birimidir. İşlemleri yönetir ve izleme ile tespit mekanizmalarını koordine eder.

IV. GERÇEKLEŞTİRİM (IMPLEMENTATION)

Sistem C++ dili kullanılarak geliştirilmiştir ve <iostream> ile <vector> standart kütüphanelerinden yararlanılmıştır.

Temel tespit mantığı:

```
if(p.getMemoryUsage() > threshold) {
    cout << "[ALERT] Suspicious
    Process Detected!";
}
```

Bu yapı, eşik değeri tabanlı basit bir anomali tespit mekanizmasını temsil etmektedir.

V. ALGORİTMA

Algoritma: İşlem Bellek İzleme ve Tespit

1. Sistem için bir bellek eşik değeri tanımla
2. İşlem listesini oluştur
3. İşlemleri sisteme ekle
4. Tüm işlemleri görüntüle
5. Ortalama bellek kullanımını hesapla
6. Her işlem için:

- Eğer bellek kullanımı > eşik → uyarı üret

7. İşlem taramasını sonlandır

VI. SONUÇLAR VE DEĞERLENDİRME

A. Simülasyon Çıktısı

```
--- Process Memory Report ---
PID: 1 | Name: chrome.exe | Memory: 450 MB
PID: 2 | Name: explorer.exe | Memory: 120 MB
PID: 3 | Name: game.exe | Memory: 800 MB
PID: 4 | Name: notepad.exe | Memory: 50 MB

Average Memory Usage: 355 MB

--- Security Scan Report ---
[ALERT] Suspicious Process Detected!
PID: 1 | Name: chrome.exe | Memory: 450 MB
[ALERT] Suspicious Process Detected!
PID: 3 | Name: game.exe | Memory: 800 MB
```

B. Analiz

Sistem şu özellikleri göstermektedir:

- ✓ Nesne yönelimli kapsülleme
- ✓ Modüler mimari
- ✓ Temel güvenlik tespit mantığı
- ✓ Görev ayrımı

Ancak bazı sınırlamalar mevcuttur:

- Gerçek işletim sistemi entegrasyonu yoktur
- Dinamik süreç takibi bulunmamaktadır
- Yapay zekâ tabanlı analiz yoktur
- Sabit eşik değerli kural sistemi kullanılmaktadır

VII. SINIRLAMALAR

Bu sistem bir simülasyondur ve gerçek işletim sistemi süreçlerine erişmemektedir. Gerçek dünya saldırı tespit sistemleri genellikle çekirdek (kernel) veya sürücü seviyesinde çalışmaktadır [2].

Ayrıca gelişmiş sistemler, daha yüksek doğruluk için kural tabanlı ve istatistiksel anomali tespit yöntemlerini birlikte kullanmaktadır [1].

VIII. GELECEK ÇALIŞMALAR

Gelecekte aşağıdaki geliştirmeler yapılabilir:

1. Sistem Seviyesi Entegrasyon
2. Makine öğrenmesi tabanlı anomali tespiti
3. Gerçek zamanlı izleme (multithreading)
4. Log kayıt sistemi
5. GUI arayüz geliştirme (Qt vb.)

C++ dili, performans ve sistem seviyesi kontrol yetenekleri nedeniyle bu geliştirmeler için uygundur [3].

IX. SONUÇ

Bu çalışmada C++ ile Nesne Yönelimli Programlama kullanılarak bir İşlem Bellek İzleme ve Güvenlik Tespit Sistemi geliştirilmiştir. Sistem, işlem takibi, bellek analizi ve anomali tespiti gibi temel güvenlik mekanizmalarını başarıyla simüle etmektedir.

Basitleştirilmiş olmasına rağmen, çalışma gerçek dünya güvenlik sistemlerinin temel yazılım mühendisliği prensipleri ile modellenebileceğini göstermektedir. Ayrıca OOP tasarımının ölçeklenebilir ve sürdürülebilir sistemler oluşturmadaki önemi vurgulanmıştır.

TEŞEKKÜR

Bu çalışmanın hazırlanmasında katkı sağlayan ders hocasına ve Kastamonu Üniversitesi'ne teşekkür edilir. Ayrıca sistemin geliştirilmesi sürecinde destek veren herkese minnettarlık ifade edilir.

KAYNAKLAR

- [1] P. Axelsson, "Intrusion Detection Systems: A Survey and Taxonomy," 2000.
- [2] D. E. Denning, "An Intrusion-Detection Model," IEEE Transactions on Software Engineering, 1987.
- [3] B. Stroustrup, The C++ Programming Language, Addison-Wesley.
- [4] Silberschatz, Galvin, Gagne, Operating System Concepts, Wiley.